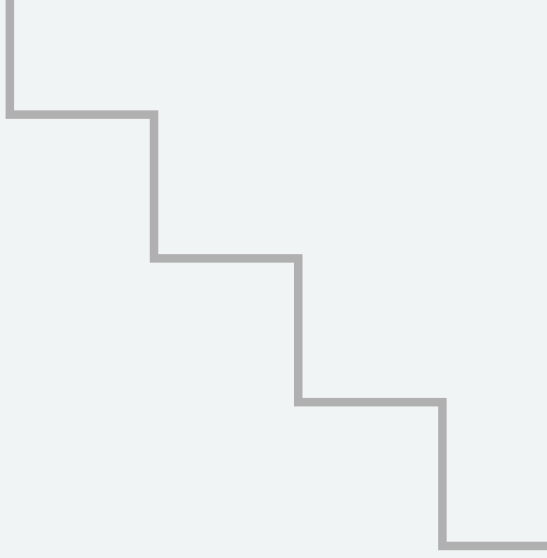


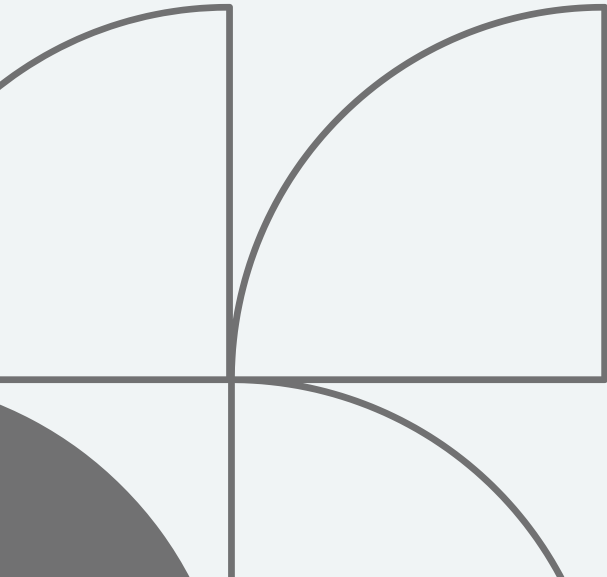
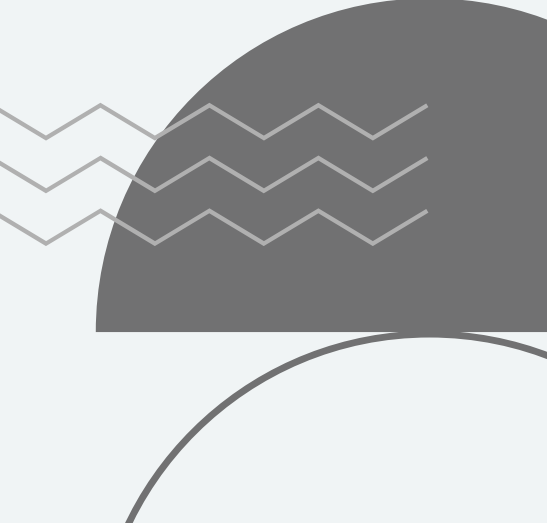


AI212 PROJECT

Weapon Detection System

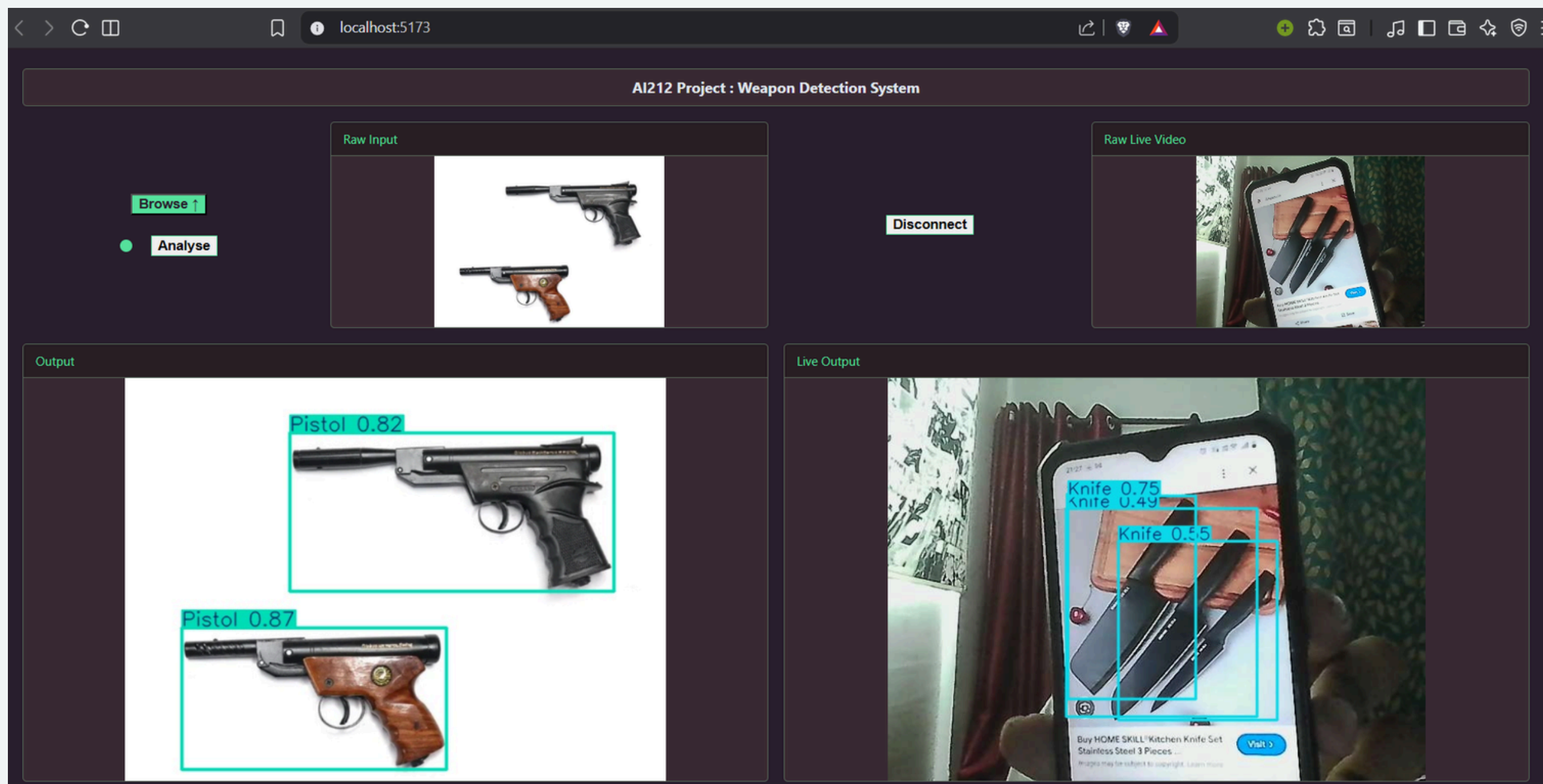
Identification of weapons in images
and videos

Team Members:

- Aditya Kumar Verma (2024AIB1002)
 - Harsh Yadav (2024AIB1006)
- 
- 

FINAL PROJECT

Upload Image to detect weapons



Connect the camera to detect any weapon live

TECHNOLOGY USED

AI / Deep Learning:

- YOLOv8 (Ultralytics): Model used for real-time object detection, chosen for its balance of speed and accuracy.
- PyTorch Backend: Acts as the underlying framework for model loading and tensor computations.

Backend:

- FastAPI
- WebSockets: Enables full-duplex communication for real-time video streaming between the client and server.

Parallel Processing:

- Ray Distributed Framework: Distributes tasks across multiple CPU/GPU workers

Frontend:

- A React-based interface that handles file uploads and manages live webcam streams using hooks to maintain a responsive, high-density UI.

WORK FLOW

The user provides an image or initiates a live camera stream via the React frontend

Input

FastAPI takes the incoming data. For live streams, this happens over WebSocket connection

Request Handling

Ray picks up the request and assigns it to an available worker node to ensure parallel execution.

Task Distribution

OpenCV draws bounding boxes and labels on the frame. The processed data (or annotated frame) is sent back to the frontend

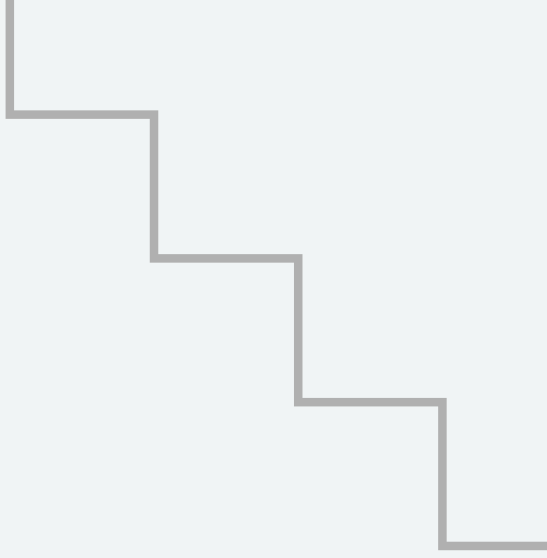
Detection

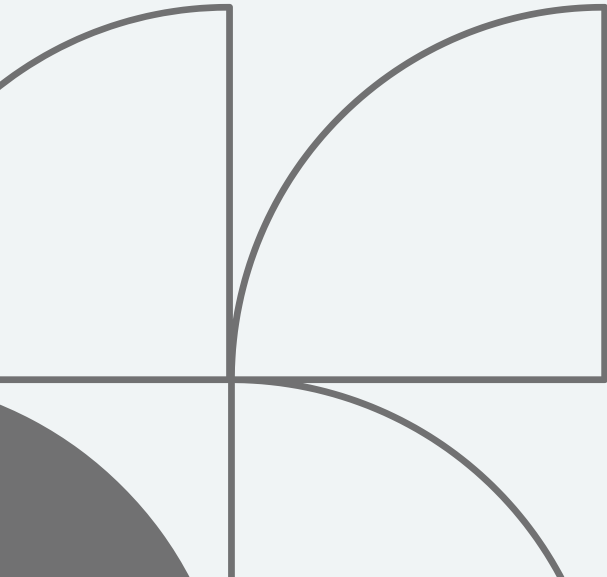
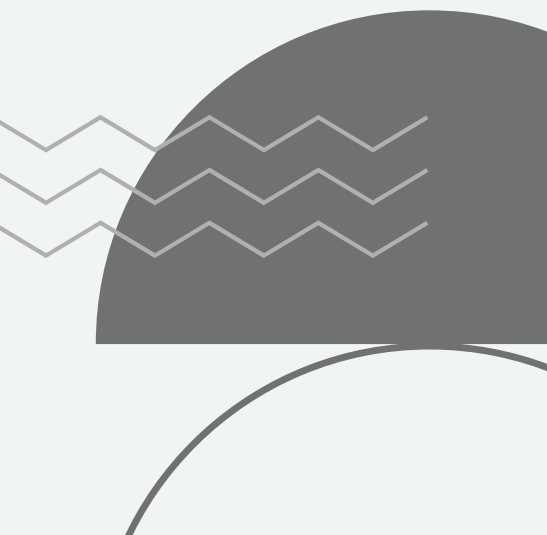
The YOLOv8 model analyzes the frame to identify potential weapons

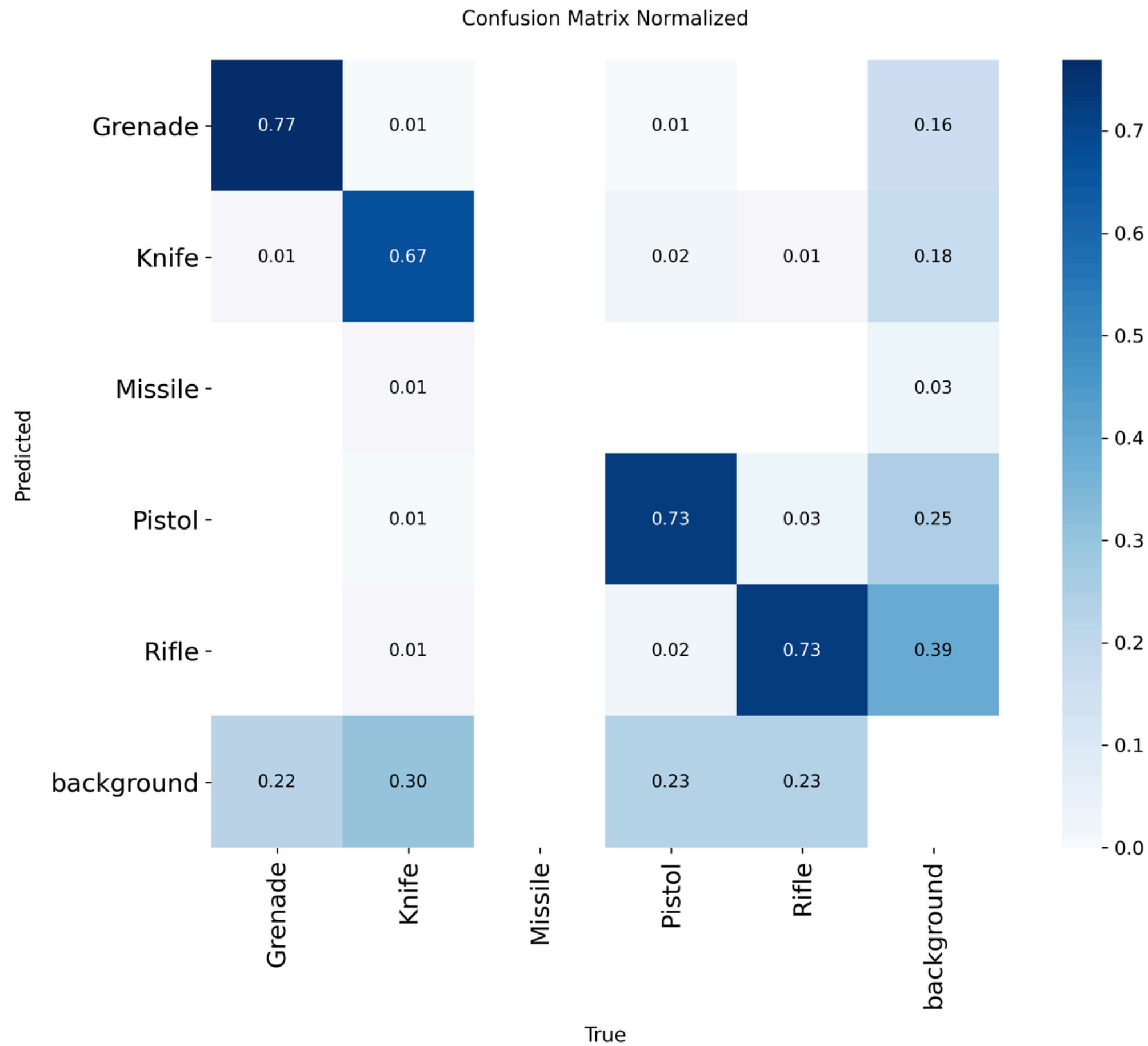
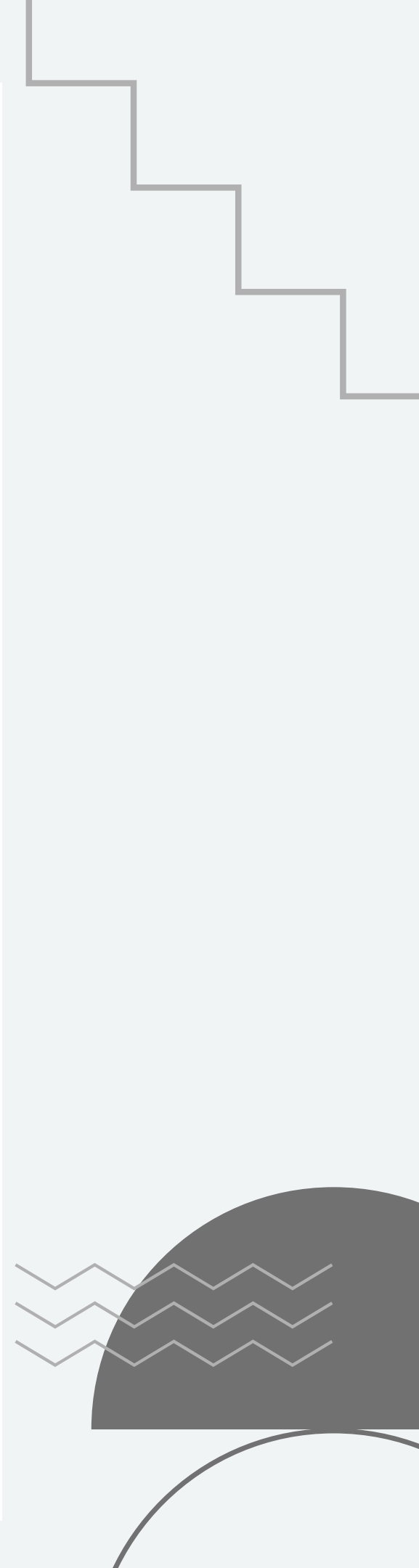
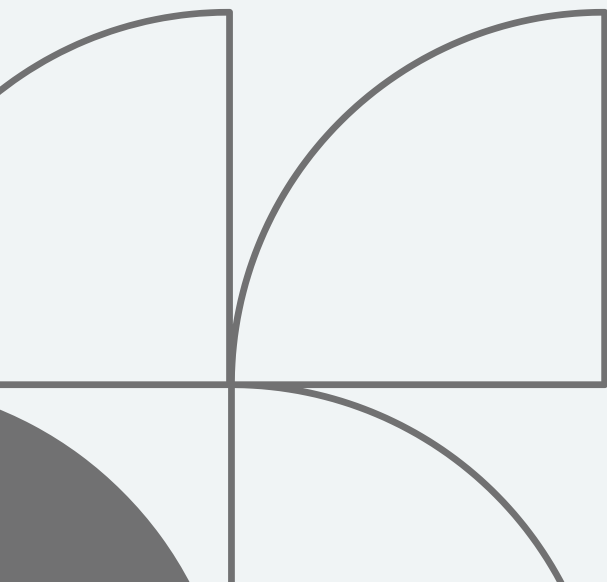
Response

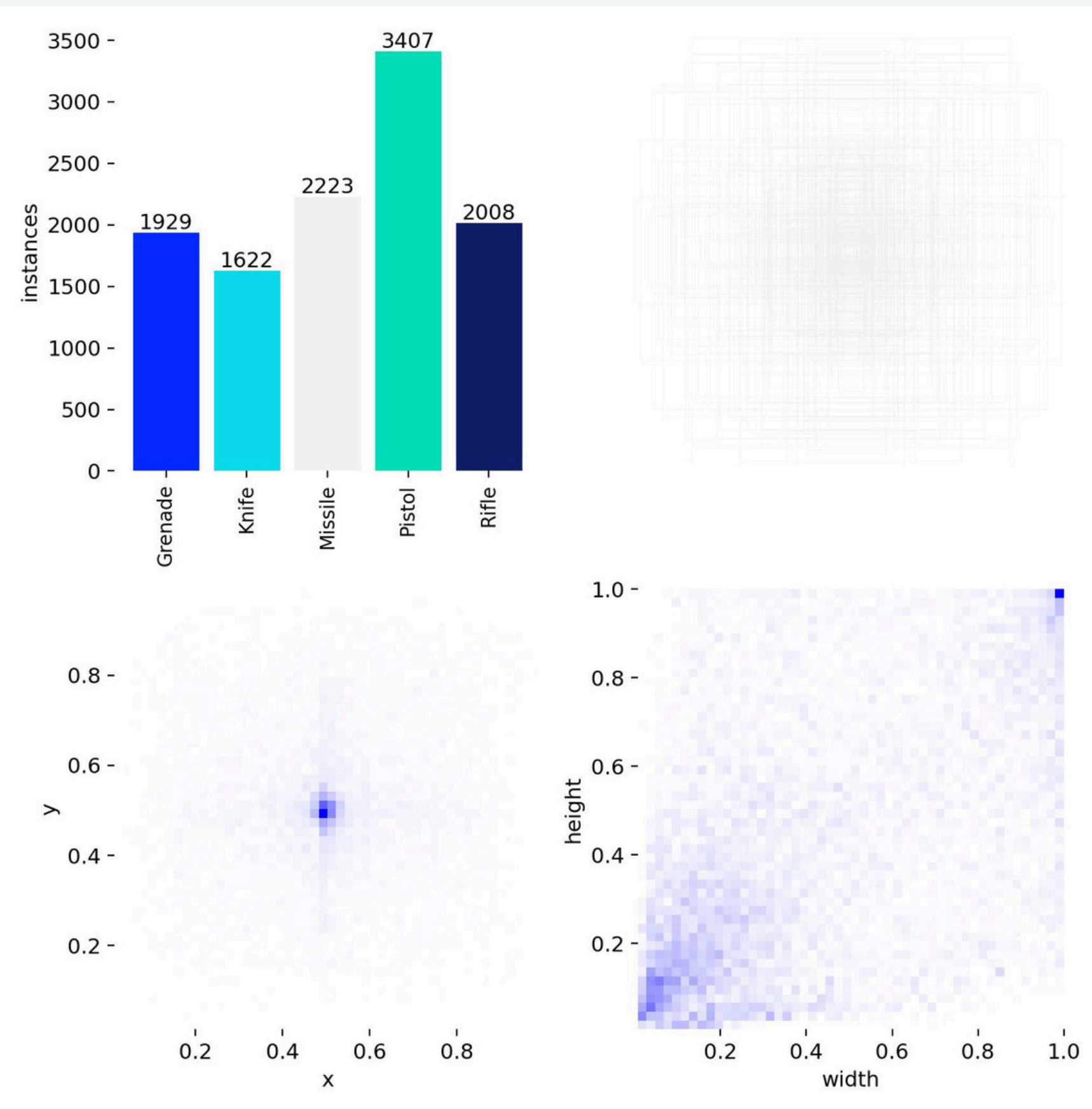


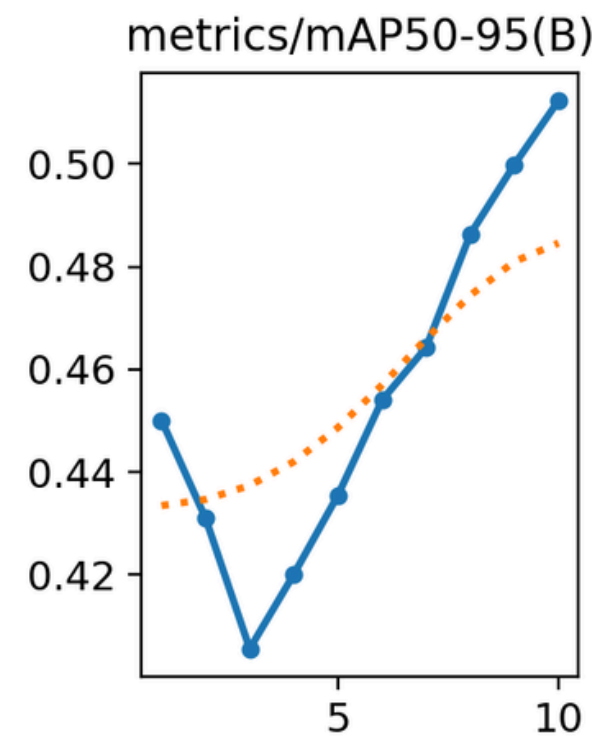
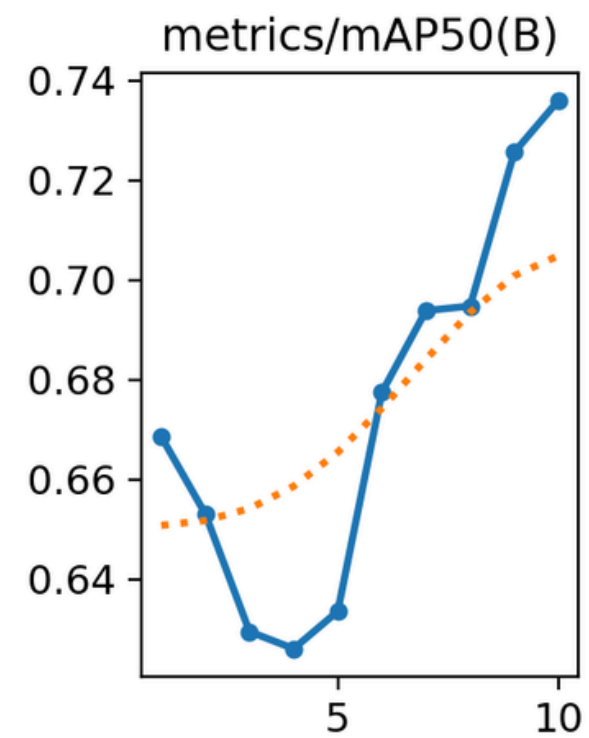
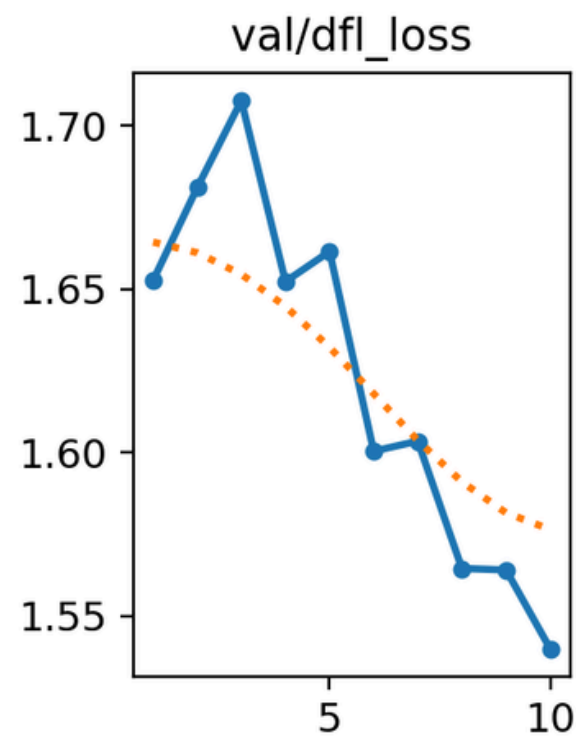
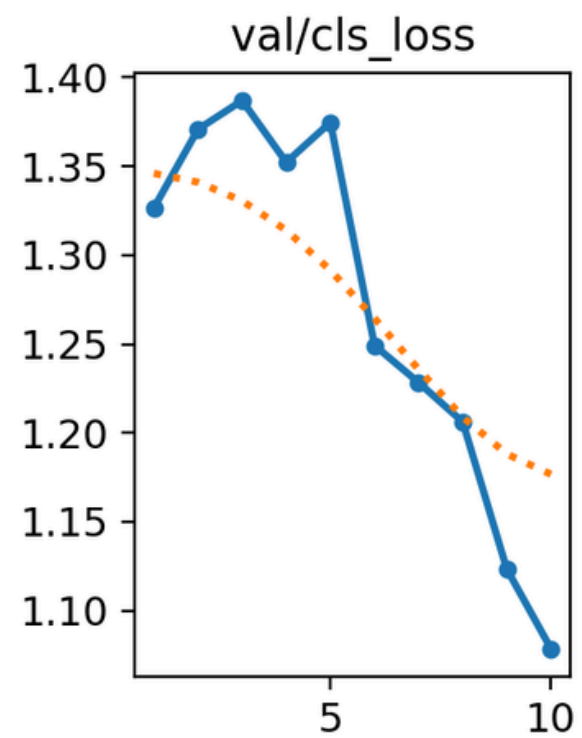
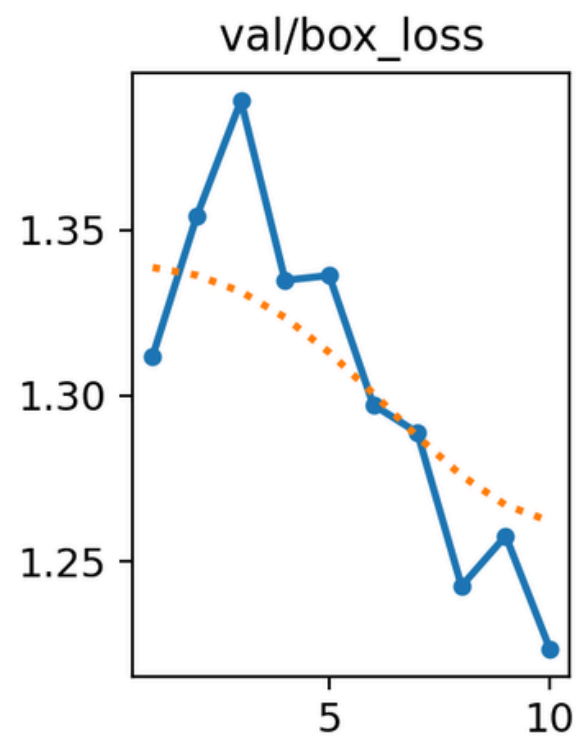
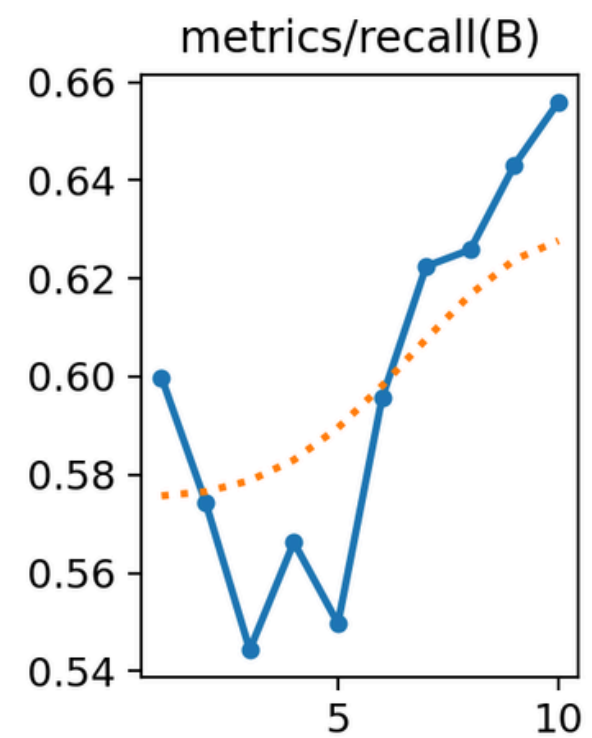
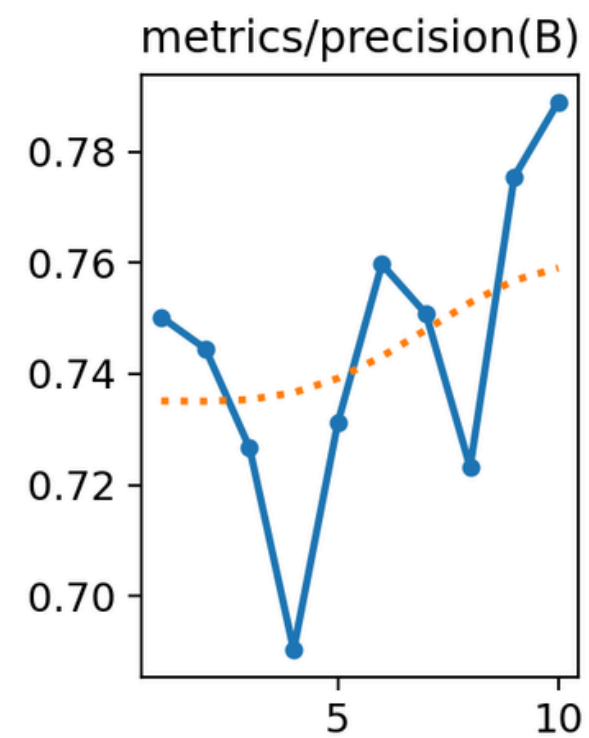
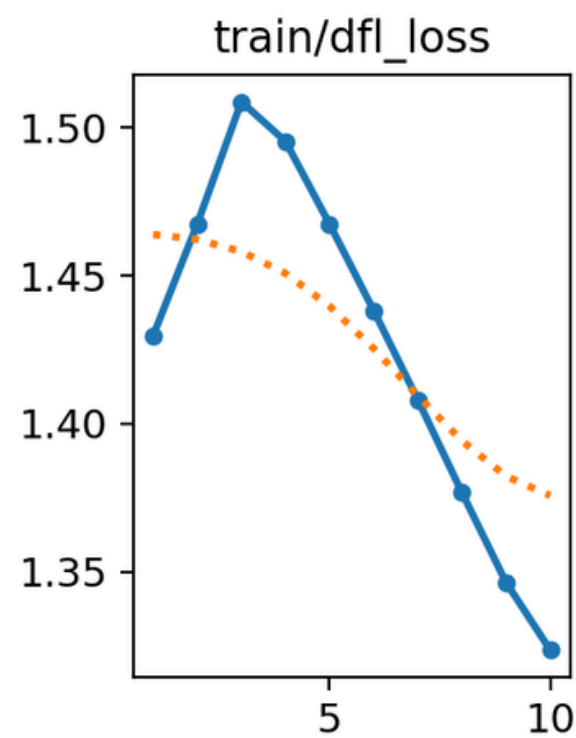
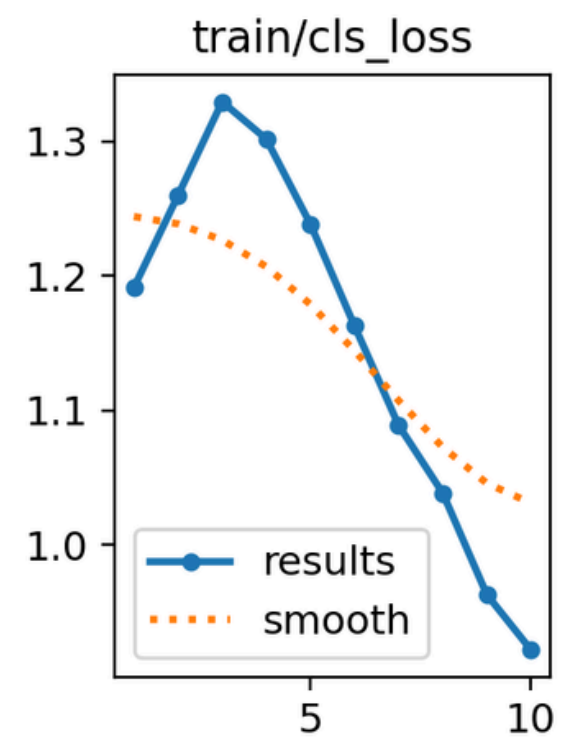
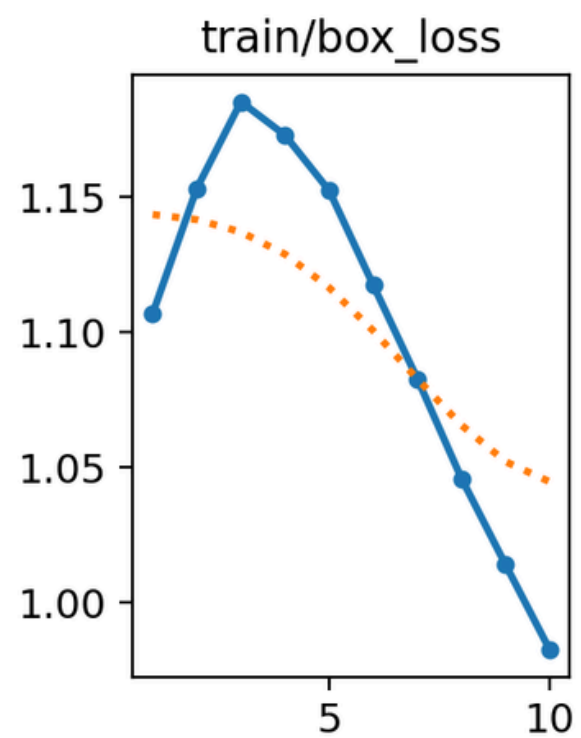
MODEL TRAINING

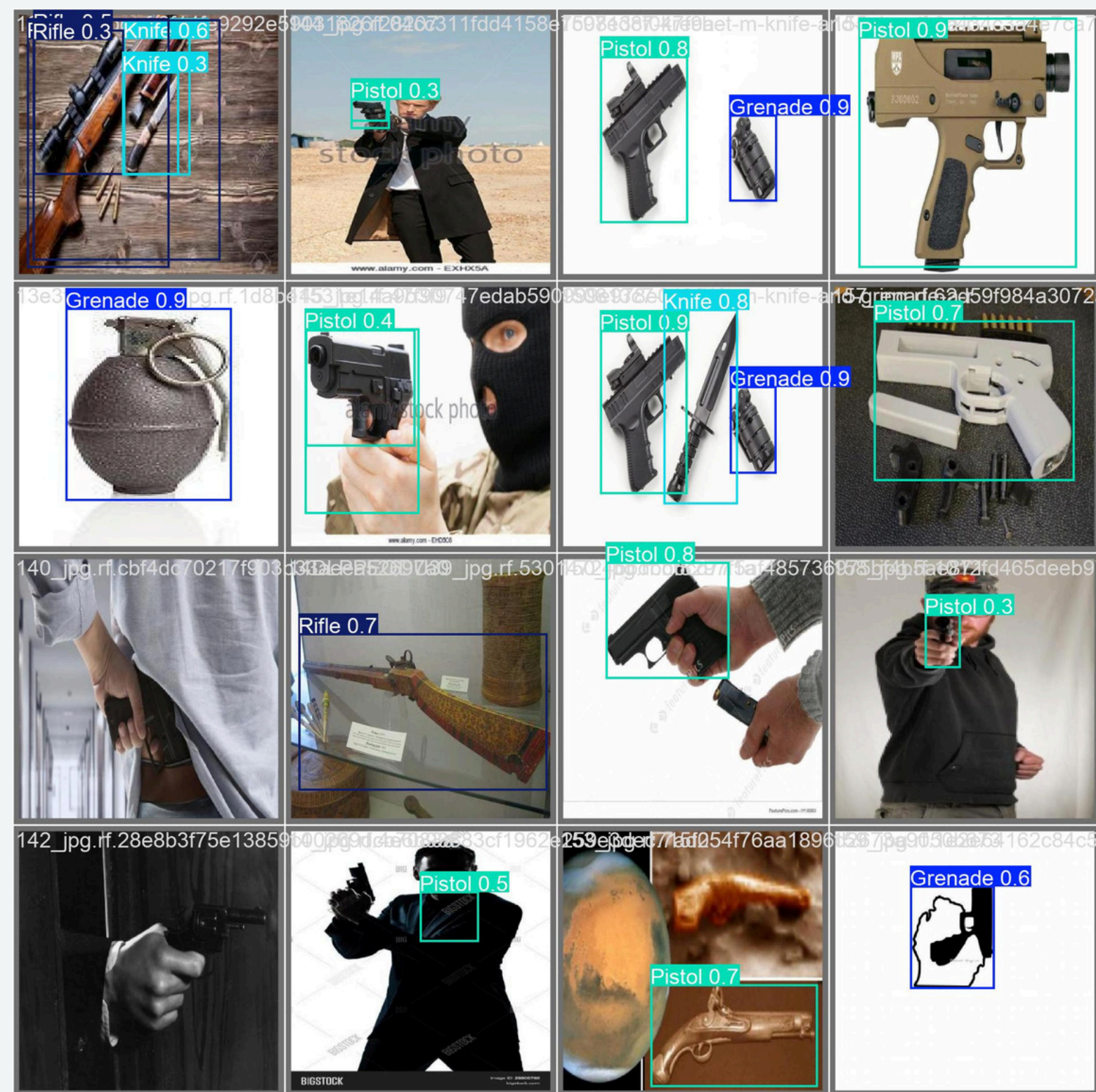
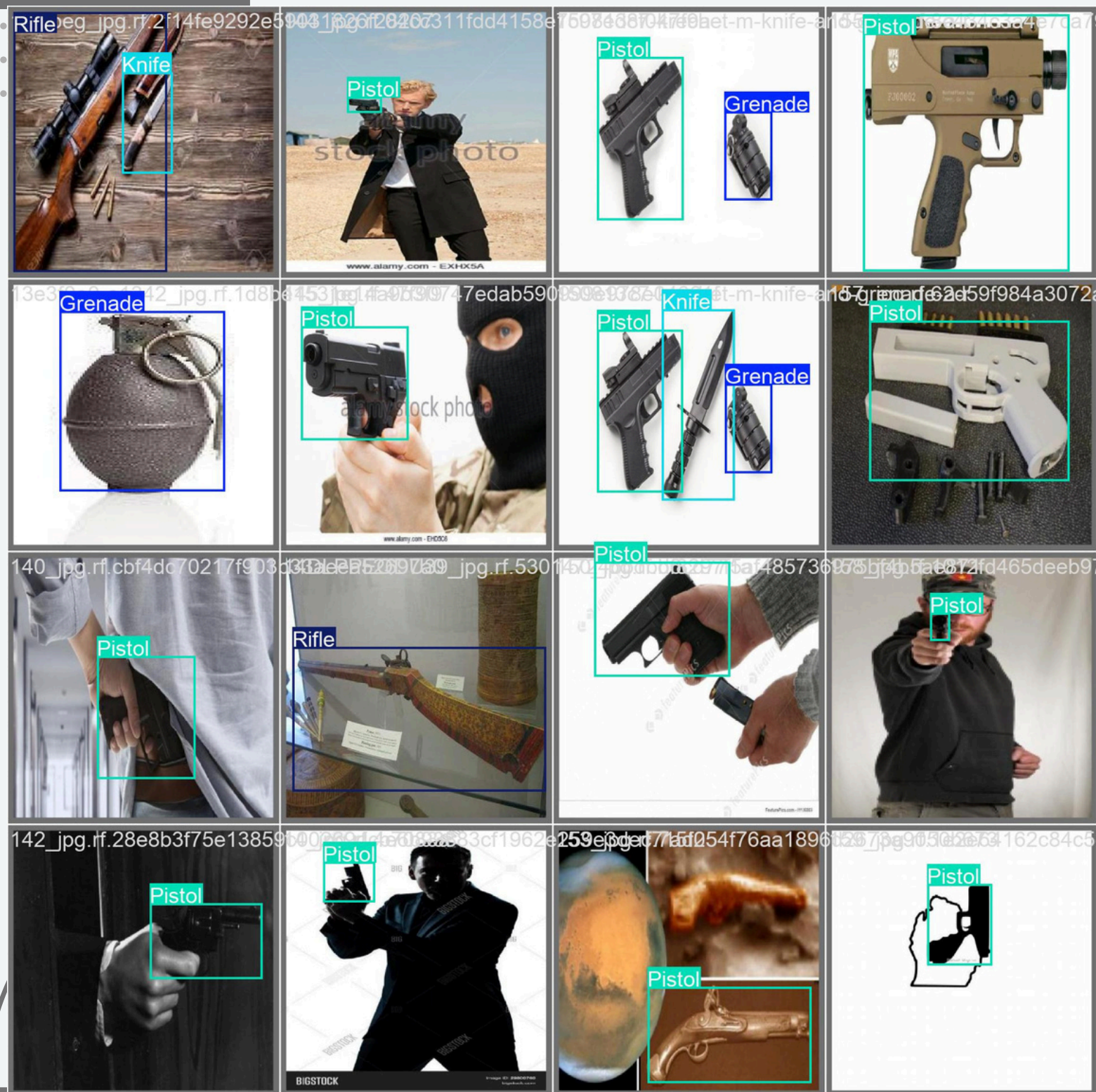


- We used a pre-built YOLO model, specifically YOLOv8 nano, and then trained on a weapon detection dataset we obtained from roboflow. The dataset contained around 9k images, with around 7k being for training and the rest for testing and validation.
 - We trained the model in two phases. For the first phase we used ray to distribute compute for the first 20 epochs. Then we fine tuned the model for 10 epochs without using ray. This approach was especially relevant on our relatively low compute power machines.
 - Upon completion of the training cycle, the model achieved a high degree of classification accuracy and localization precision. The graphs for different types of losses over epochs for the second training cycle and a sample output are shown in the next few slides.
- 
- 









BACKEND FUNCTIONS

```
@app.post("/detect")
async def detect_image(file: UploadFile = File(...)):
    image_bytes = await file.read()
    selected_worker = next(worker_cycle)
    ref = selected_worker.process_image.remote(image_bytes)
    annotated_image_bytes = await ref
    return Response(content=annotated_image_bytes, media_type="image/jpeg")

@app.post("/video")
async def detect_video(file: UploadFile = File(...)):
    input_path = os.path.join(TEMP_DIR, f"in_{file.filename}")
    output_path = os.path.join(TEMP_DIR, f"out_{file.filename}")
    with open(input_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)
    selected_worker = next(worker_cycle)
    await selected_worker.process_video.remote(input_path, output_path)
    os.remove(input_path)
    return FileResponse(output_path, media_type="video/mp4")

@app.websocket("/ws/live")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    session_worker = next(worker_cycle)
    try:
        while True:
            bytes_data = await websocket.receive_bytes()
            processed_bytes = await session_worker.process_stream_frame.remote(bytes_data)
            await websocket.send_bytes(processed_bytes)
    except WebSocketDisconnect:
        print("Live stream disconnected.")
```

LIVE VIDEO DETECTION

01	Capture	Camera frames are captured on the client side and sent as binary bytes through the WebSocket.
02	Ingestion	The backend receives these bytes and converts them into a format compatible with the model (using NumPy).
03	Inference	YOLO predicts the presence of weapons in milliseconds.
04	Annotation	Bounding boxes are overlaid onto the frames locally on the server.
05	Streaming	The processed frames are pushed back to the frontend, where the video feed is updated instantly.

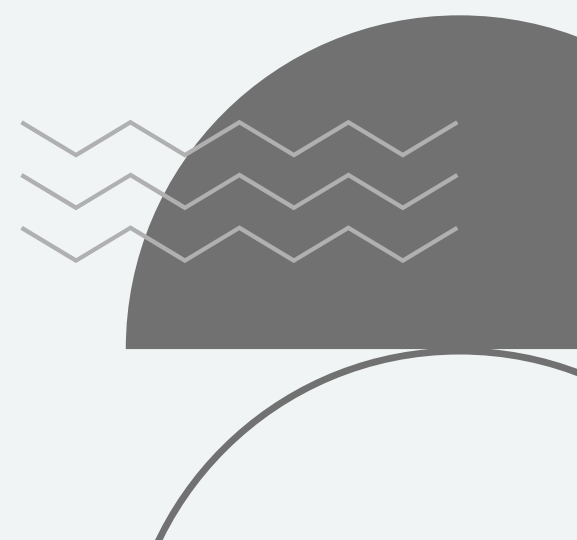
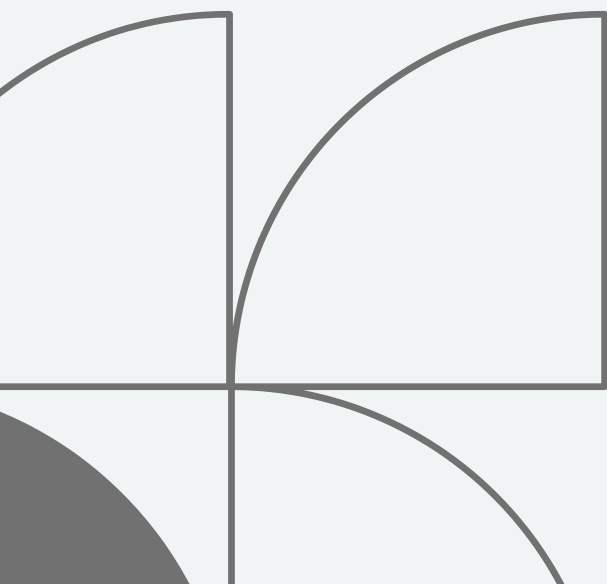


PERFORMANCE OPTIMIZATIONS

Multiple Ray Workers: By using a distributed actor model, the system can handle multiple concurrent users or high-resolution streams without crashing.

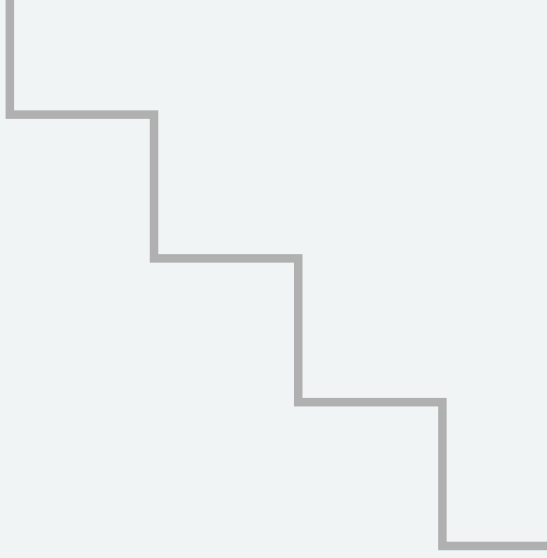
Confidence Thresholds: Fine-tuned logic that ignores low-probability detections, significantly reducing false positives in complex environments.

Dockerization: The entire backend and worker environment is containerized, ensuring consistent performance across different hardware and cloud environments.





FUTURE IMPROVEMENTS



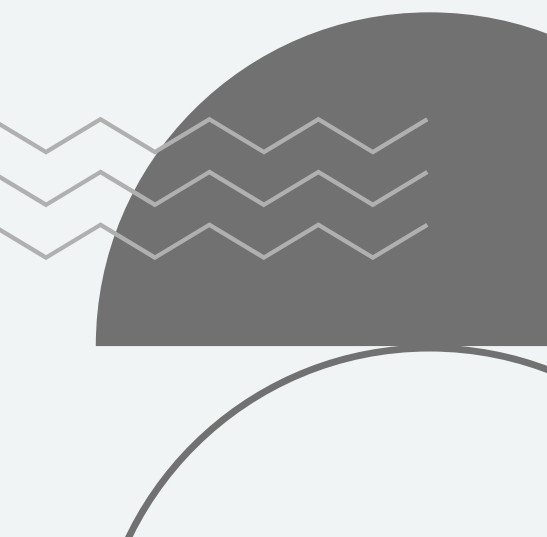
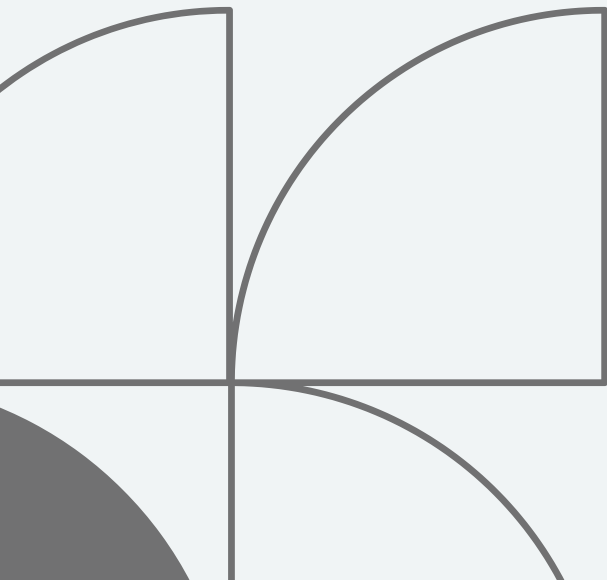
Advanced Classification: Expanding the model to distinguish between different types of weapons (e.g., firearms vs. knives).

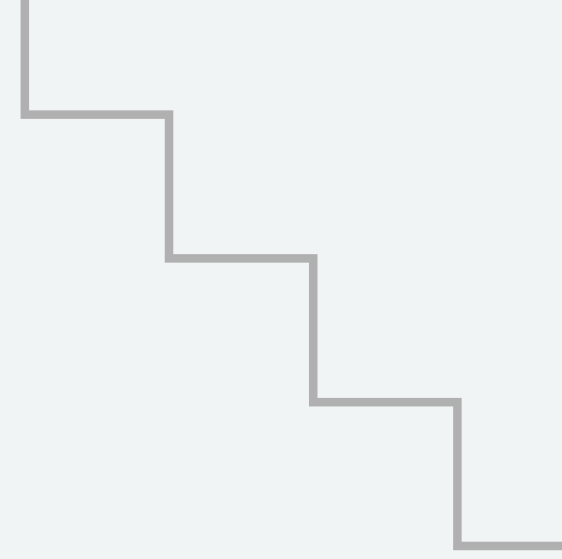
Automated Alerts: Integrating an instant notification system (via Email or SMS) when a high-confidence threat is detected.

Expanded Recognition: Adding face recognition to identify individuals in the vicinity of a detected threat.

Deployment Versatility: Moving beyond local servers to Cloud and Edge AI (like NVIDIA Jetson) for on-site, low-power monitoring.

Mobile Support: Developing a dedicated mobile application for security personnel to receive real-time feeds on the go.





**THANK
YOU**

